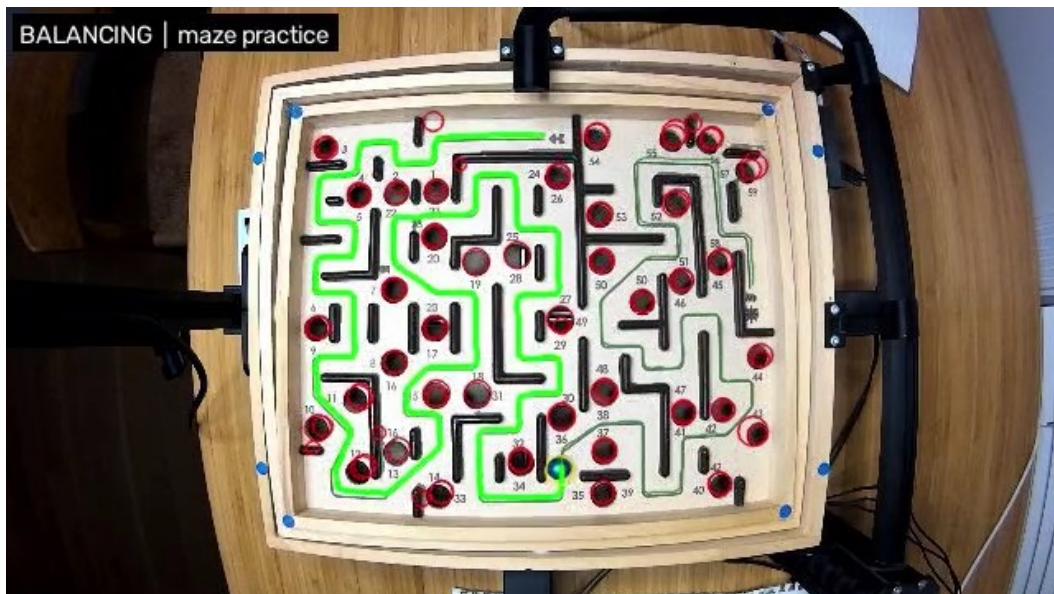


CyberRunner: ODIL vs reinforcement learning

A visual guide to the methods, the equations and the experiments

Ball-on-plate and the labyrinth on a real tilting-plate robot. What each method is, exactly how it learns, what it has achieved so far, and the experiment now running: training ODIL, SAC and PPO from real rig data only.

Part	Question	Status
1. Ball on plate, sim -> rig	Train in a simulator, run on the real plate	done: ODIL = RL accuracy, 2.5x smoother
2. Maze	Follow the printed route of the real board	done: real maze solved 3 times
3. Ball on plate, rig only	How much real rig time does each method need?	running: dry test passed



The rig's live view during the first complete run of the real labyrinth (route already done in green).

1. The rig and the control loop

A camera looks down on a plate that two motors tilt. Thirty times a second the software finds the plate and the ball, decides how to tilt, and moves the motors. Every method in this guide only changes step 4: the controller. Everything else is shared.

Thirty times a second: see the ball, decide a tilt, move the motors



Figure 1. The control loop. Steps 1-3 and 5-6 are identical for all methods.

Measuring the state

Four markers on the plate give its tilt; the ball's colour gives its pixel, which is back-projected onto the plate plane in millimetres. Speed comes from two consecutive photos:

$$\mathbf{v}_n = \frac{\mathbf{p}_n - \mathbf{p}_{n-1}}{t_n - t_{n-1}}$$

Speed from the camera. It is noisy: a resting ball seems to jiggle by up to 18 mm/s.

The tilt servo

The motors do not hold an angle reliably (the linkage slips), so the camera closes the loop: each frame the motor target moves by the change in the wanted tilt plus a correction on the measured error, compared with the target from when the frame was taken ($D = 4-5$ frames ago):

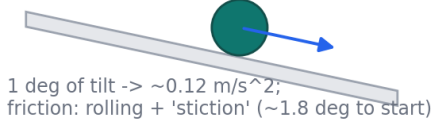
$$m_{n+1} = m_n + c \left[(\theta_n^* - \theta_{n-1}^*) + g(\theta_{n-D}^* - \hat{\theta}_n) \right], \quad g = 0.15$$

m = motor position (ticks), c = ticks per degree, θ^* = wanted tilt, $\hat{\theta}$ = camera-measured tilt. Safety limits: no push beyond 7 deg, and the motor stays inside a fixed travel cap.

2. The physics every method lives with

Rolling ball on a tilted plate

$$\text{acceleration} = 5/7 g \sin(\text{tilt})$$



The real plate follows the command late (delay + lag)

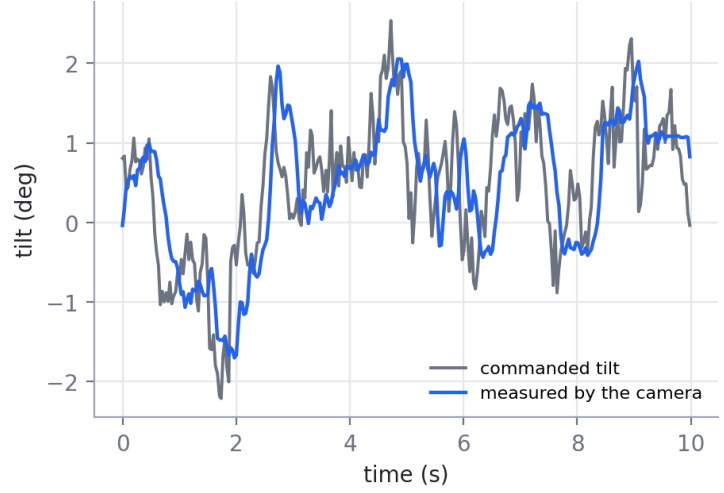


Figure 2. Left: a rolling ball accelerates down the slope. Right: real rig data (dry run) -- the camera-measured tilt (blue) follows the command (grey) with a delay and a lag.

Rolling

$$\ddot{\mathbf{p}} = \frac{5}{7} g \sin \theta \approx k(\theta + \mathbf{b}), \quad k = \frac{5}{7} g \frac{\pi}{180} \approx 0.12 \text{ m s}^{-2} \text{ deg}^{-1}$$

A solid ball rolling without slipping accelerates at 5/7 of the free-sliding value. $\mathbf{b}$ is a small level error of the plate.

Friction and stiction

Two friction effects matter at these small angles: rolling friction a_r , and stiction -- a resting ball does not move until the tilt exceeds a breakaway level a_s (about 1.8 deg). Both are written smoothly so that gradients exist:

$$\mathbf{a} = \mathbf{d} - w(\|\mathbf{v}\|) h(\|\mathbf{d}\|) \frac{\mathbf{d}}{\|\mathbf{d}\|} - (1 - w(\|\mathbf{v}\|)) a_r \frac{\mathbf{v}}{\|\mathbf{v}\|}, \quad \mathbf{d} = k(\theta + \mathbf{b})$$

w blends from 'resting' to 'rolling' as the speed grows; h is a soft minimum, so a drive below the breakaway is cancelled.

$$w(s) = e^{-(s/v_s)^2}, \quad h(x) = -\kappa \log(e^{-x/\kappa} + e^{-a_s/\kappa}) \approx \min(x, a_s)$$

$v_s = 1 \text{ cm/s}$, κ sets how sharp the soft minimum is.

The plate is late

$$\dot{\theta}(t) = \frac{\mathbf{u}(t-d\Delta t) - \theta(t)}{\tau}, \quad \Delta t = \frac{1}{29} \text{ s}, \quad d \in \{1, 2, 3\}, \quad \tau \approx 0.03 - 0.1 \text{ s}$$

The plate follows the command after a pure delay of d frames and a first-order lag τ . This delay is the main reason fast controllers overshoot.

Fitting the physics to rig data (system identification)

ODIL needs these numbers for this particular rig. They are measured from recordings by least squares: the gain k and rolling friction from measured accelerations, the delay and lag from how the measured tilt follows the command:

$$\hat{k}, \hat{a}_r = \arg \min_{k, a_r} \sum_n \left\| \mathbf{a}_n^{\text{meas}} - k \theta_n + a_r \frac{\mathbf{v}_n}{\|\mathbf{v}_n\|} \right\|^2, \quad \hat{d}, \hat{\tau} = \arg \min_u \sum_n (\theta_n^{\text{meas}} - \theta_n^{\text{model}}(u; d, \tau))^2$$

Fitted on the rig: $k \sim 0.08\text{-}0.11 \text{ m/s}^2 \text{ per degree}$, $a_r \sim 0.03\text{-}0.05 \text{ m/s}^2$, delay 3-4 frames, lag 0.03-0.1 s.

3. ODIL -- Optimizing a Discrete Loss

Idea in one sentence: write the physics as a penalty, make the whole trajectory and every command unknowns, and solve for all of them at once by gradient descent; then fit a small network to reproduce the commands from the ball's state.

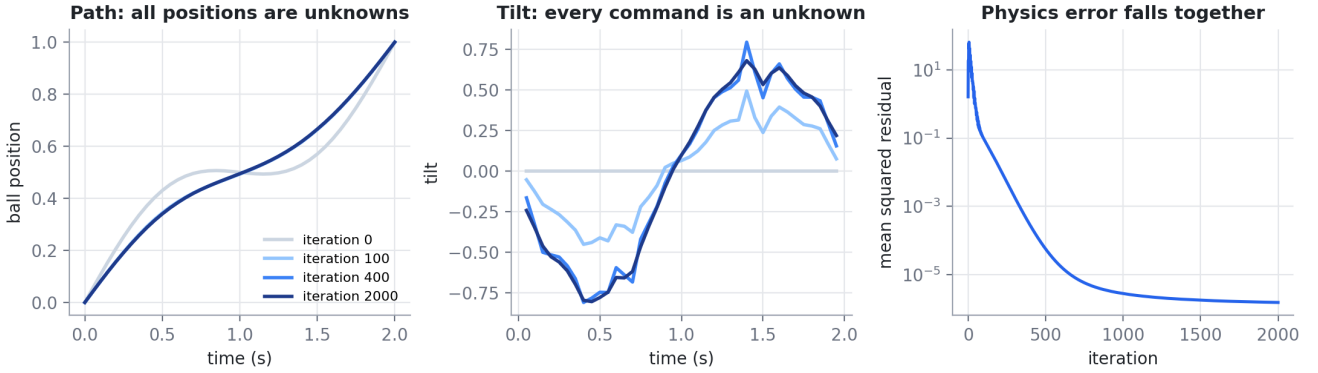


Figure 3. A worked ODIL example (1-D). Start from a wrong guess (grey): every position and every tilt is adjusted together until each step obeys the physics (right: the physics error falls by six orders of magnitude) while the ball goes from rest at 0 to rest at 1.

The loss

$$L(\mathbf{x}, \mathbf{u}, \phi) = \sum_n \left\| \mathbf{x}^{n+1} - \mathbf{x}^n - f(\mathbf{x}^{n+\frac{1}{2}}, \mathbf{u}^{n+\frac{1}{2}}) \Delta t \right\|^2 + \lambda \sum_n \left\| \mathbf{p}^n - \mathbf{r}^n \right\|^2 + \mu \sum_n \left\| \mathbf{u}^{n+1} - \mathbf{u}^n \right\|^2 + \nu \sum_n \left\| \mathbf{u}^n - \pi_\phi(\mathbf{x}^n) \right\|^2$$

Term 1: physics residual at every step (midpoint rule). Term 2: tracking the target or reference $\mathbf{r}$. Term 3: smooth commands. Term 4: the commands must be reproducible by the policy network π .

$$(\mathbf{x}, \mathbf{u}, \phi) \leftarrow (\mathbf{x}, \mathbf{u}, \phi) - \eta \nabla L \quad (\text{all states, all commands and the policy network together})$$

One gradient step updates everything. Thousands of trajectories (random starts, targets, physics parameters) are optimised together, so the network learns a general controller.

Closed-loop refinement

A policy fitted to optimised trajectories never saw its own compounding errors. It is therefore refined by back-propagating through closed-loop rollouts of a differentiable model with the rig's delay, lag, camera noise and friction:

$$J(\phi) = \mathbb{E} \left[\sum_n \left(\frac{\|\mathbf{p}_n - \mathbf{r}_n\|}{5 \text{ mm}} \right)^2 + w_j \|\Delta \mathbf{a}_n\|^2 + w_e \left\| \frac{\mathbf{u}_n - \dot{\mathbf{r}}_n/k}{u_{\max}} \right\|^2 \right], \quad \mathbf{u}_n = \pi_\phi(\text{obs}_n), \quad \mathbf{p}_{n+1} = \text{sim}(\mathbf{p}_n, \mathbf{u}_n)$$

Tracking error (in units of 5 mm), smoothness, and tilt beyond what a perfect follower would need. The gradient flows exactly through the physics -- this is what RL has to estimate from samples.

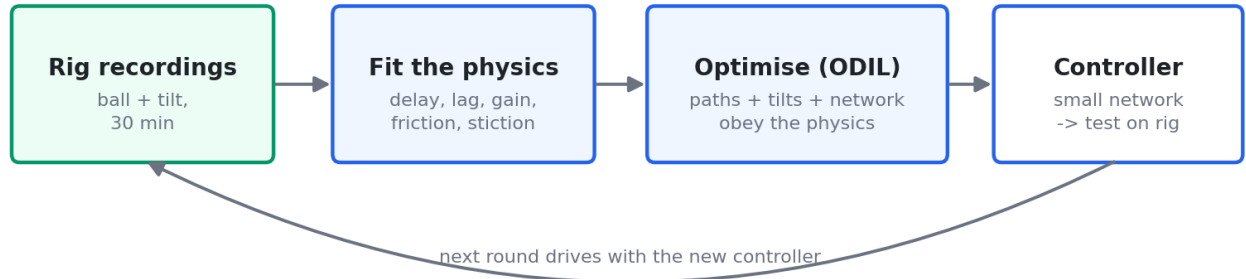


Figure 4. ODIL trained only from rig data: record, fit the physics, optimise, test; the next round records with the new controller.

4. Reinforcement learning: what PPO and SAC share

RL never sees the physics. It tries tilts, receives a reward, and gradually prefers actions that led to more reward. Both RL methods here use the same observation (target error, velocity, position, tilt, target radius, the last 4 commands), the same rate limit and the same reward:

$$r_t = -\frac{d_t}{0.1\text{m}} + 1[d_t < R] + 0.5 1[d_t < R \wedge \|\mathbf{v}_t\| < 1 \text{ cm/s}] - 0.3 \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2 - 0.01 \|\mathbf{a}_t\|^2 - 0.5 1[\text{at frame}]$$

Closer is better; a bonus inside the target and more when also still; small costs for jerky and large tilts and for touching the frame.

$$G_t = \sum_{l=0}^{\infty} \gamma^l r_{t+l}, \quad \gamma = 0.99$$

RL maximises the discounted sum of future rewards -- it has to discover which early actions lead to good outcomes seconds later.

PPO -- Proximal Policy Optimization (on-policy)

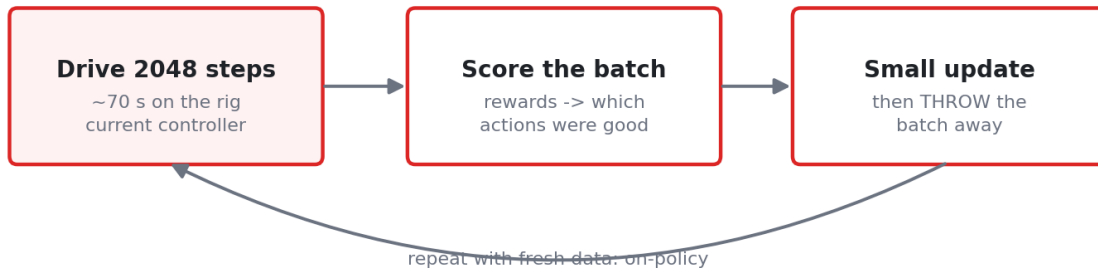


Figure 5. PPO drives a batch with the current controller, learns from it once, then discards it.

$$L^{\text{CLIP}}(\phi) = \mathbb{E}_t [\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)], \quad \rho_t = \frac{\pi_\phi(a_t|s_t)}{\pi_{\phi_{\text{old}}}(a_t|s_t)}, \quad \epsilon = 0.2$$

rho compares the new and old policy; clipping keeps each update small and stable.

$$\hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad \lambda = 0.95$$

The advantage A-hat says how much better an action was than expected; V is a learned value function.

PPO in simulation: how much experience it needed

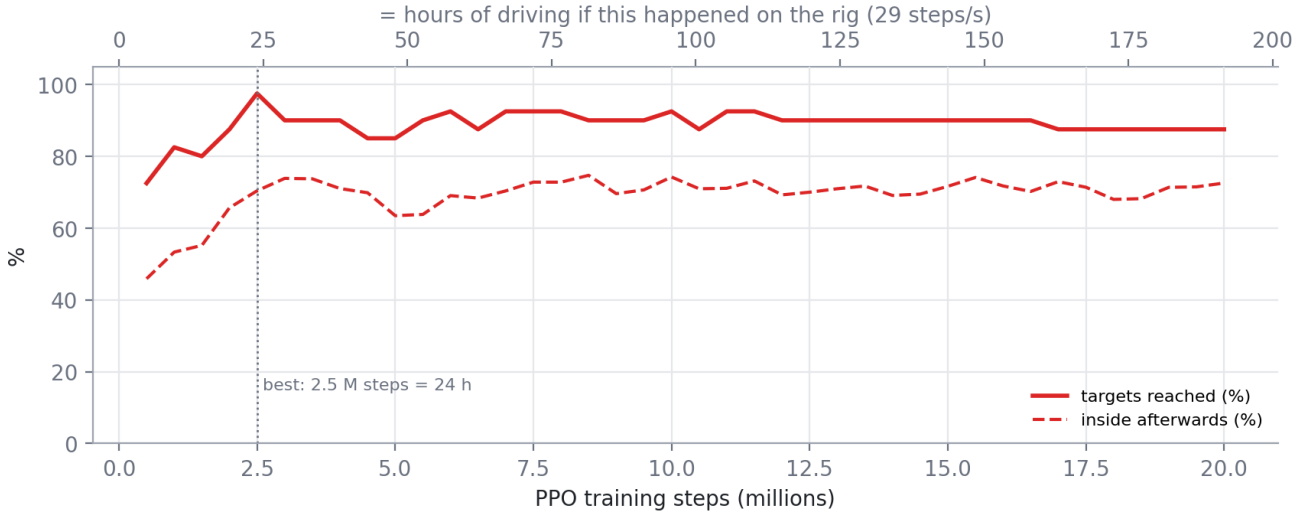


Figure 6. PPO's real learning curve in simulation (v3). Its best version needed 2.5 million steps -- 24 hours of driving if it had been learned on the rig.

5. SAC -- Soft Actor-Critic (off-policy)

SAC keeps every step it has ever driven in a memory and learns from it again and again. A critic learns how good each tilt is in each situation; the actor (the controller) moves towards what the critic rates highly. 'Soft' means it is rewarded for staying a little random, which keeps it exploring.

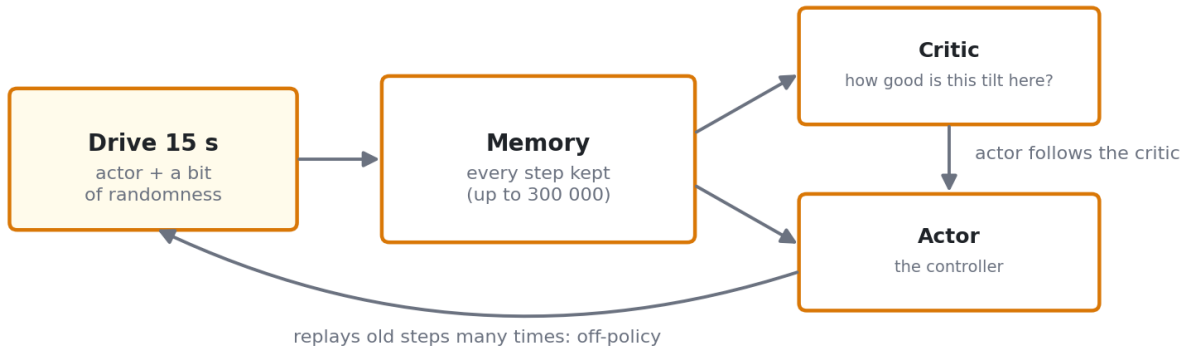


Figure 7. SAC: drive, store, and replay old steps many times. Much less driving than PPO for the same learning.

$$J(\pi) = \mathbb{E} \left[\sum_t \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right], \quad \mathcal{H} = -\mathbb{E}_{a \sim \pi} [\log \pi(a|s)]$$

The objective: reward plus an entropy bonus $\alpha * H$ (how random the policy still is).

$$L_Q = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q(s,a) - r - \gamma \left(\min_{i=1,2} \bar{Q}_i(s',a') - \alpha \log \pi(a'|s') \right) \right)^2 \right], \quad a' \sim \pi(\cdot|s')$$

Critic: match the reward plus the discounted value of the next state; two critics, take the smaller (against over-optimism); Q-bar are slowly updated target copies.

$$L_\pi = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi} \left[\alpha \log \pi(a|s) - \min_{i=1,2} Q_i(s,a) \right]$$

Actor: choose actions the critic rates highly while staying somewhat random.

On the rig, SAC's network updates happen between 15-second episodes with the plate held level, so learning never slows the 29 Hz control loop.

The three methods in one line each

	Learns from	Needs a model	Experience reuse	Rig time (expected)
ODIL	physics fitted to rig data	yes (fitted)	-- (optimises through it)	~30 min per round
SAC	rewards on real attempts	no	replays everything	~2-5 h
PPO	rewards on real attempts	no	uses each batch once	~10-30 h

6. What the controllers actually do

Same 4 starts (dots), same targets (red): paths of the trained controllers (simulation)

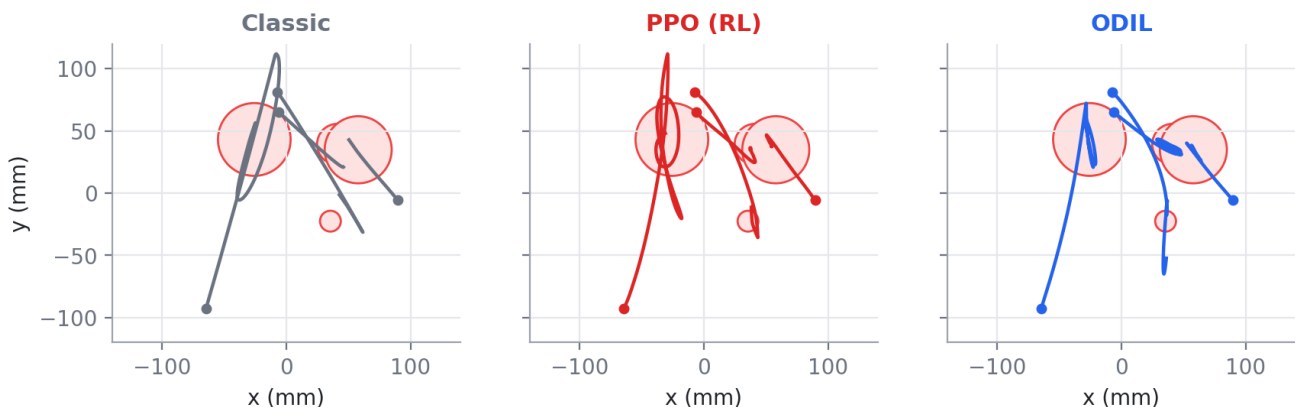


Figure 8. Simulated paths of the trained controllers from the same four starts to the same targets. Classic hesitates and stops short; PPO arrives fast but loops and corrects in bursts; ODIL arrives smoothly but can overshoot once (lower path).

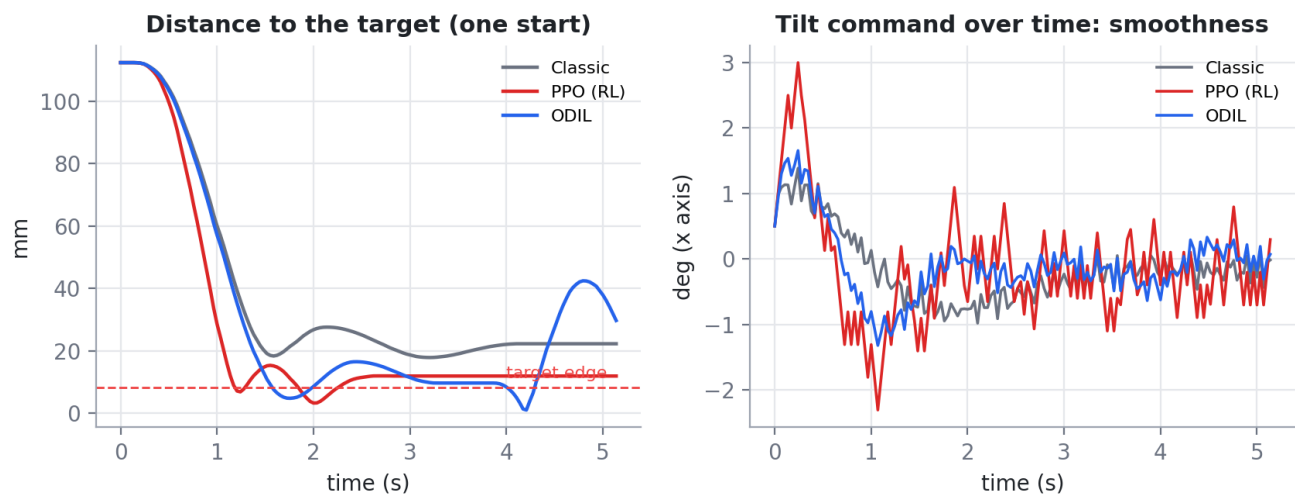


Figure 9. One start in detail. Left: distance to the target (dashed = target edge); in this run ODIL later drifts out again -- the stiction problem that the friction compensation addresses. Right: the tilt commands -- PPO's are visibly jerky, ODIL's smooth.

Results so far (part 1)

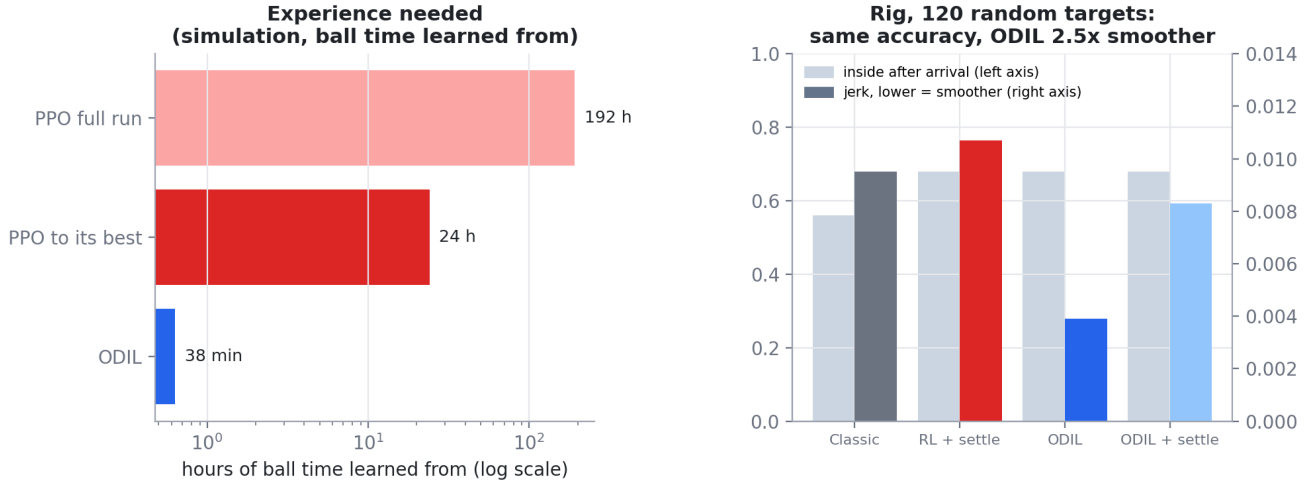


Figure 10. Left: experience each method learned from in simulation. Right: on the real rig (120 random targets, 30 per setup) ODIL is as accurate as RL and about 2.5 times smoother.

$$\text{jerk} = \frac{1}{N} \sum_n \|\mathbf{a}_n - \mathbf{a}_{n-1}\|^2, \quad \text{inside} = \frac{1}{T} \int_{t_{\text{in}}}^{t_{\text{in}} + 4\text{s}} 1[d(t) < R] dt$$

The two headline metrics. 'Inside' = share of the 4 s after arrival spent inside the target.

7. The experiment now: training from real rig data only

No hand-made simulator: every method learns only from what the real rig shows it. One process holds the rig for a whole session (restarts froze the camera), and every controller is tested on the same 30 fixed targets, logged with the rig minutes it has used so far. Result: learning curves of quality against real rig time.

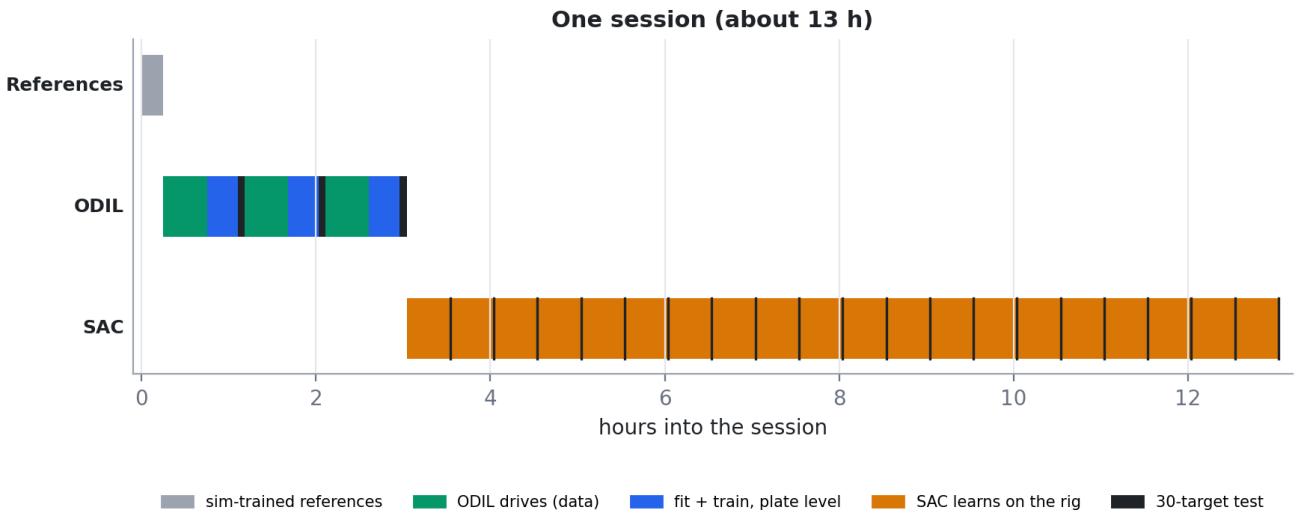


Figure 11. One session: references, three ODIL rounds (drive / fit + train / test), then 10 hours of SAC with a test every 30 minutes.

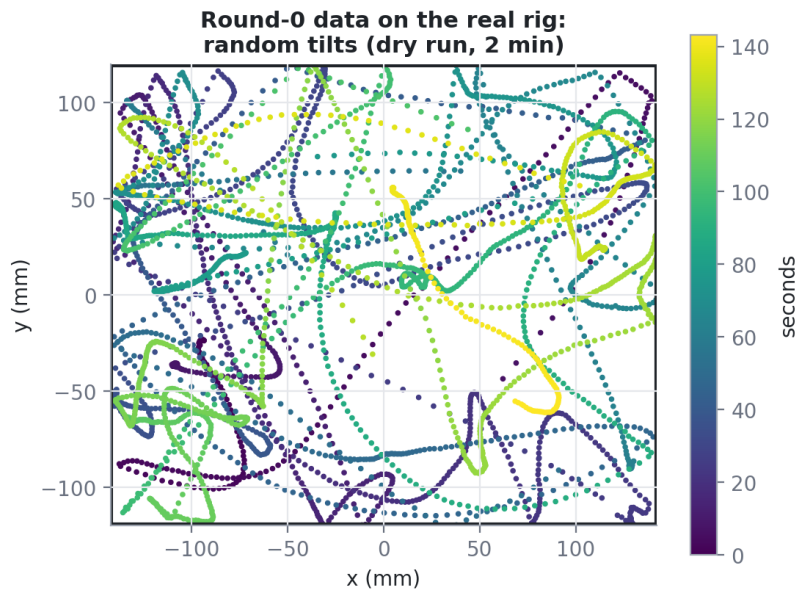


Figure 12. ODIL's round-0 data from the dry test: random tilts, no controller. The ball covered the whole plate -- too fast (it was lost ~10 % of frames); the real run uses gentler tilts (max 1.75 deg, braking above 15 cm/s).

Dry test, all phases (2-4 minutes each -- checks the pipeline, not quality)

Run (dry test)	Rig data	Reached	Time to reach	Inside after	Final dist.	Jerk
ref_odil_v11_sim	0 min	6/6	1.4 s	58 %	11 mm	0.0036
ref_ppo_v3_sim	0 min	5/6	1.2 s	92 %	8 mm	0.0184
odil	2 min	0/6	-	0 %	65 mm	0.0015
sac	2 min	1/6	0.4 s	3 %	74 mm	0.0002
sac	4 min	2/6	5.8 s	5 %	83 mm	0.0126

PPO: the motor watchdog stopped the run (motor 1 command 6317 left +-1300 of 4878) -- the safety stop worked; cap tightened to 1000 ticks for the real runs

Plan for the paper

- **References** each session: the simulation-trained ODIL v11 and PPO v3 on the same targets (drift anchor; links part 1 and part 3).
- **ODIL x 3 runs, SAC x 3 runs** (up to 10 h each), **PPO x 1 run** (up to 30 h), order alternated across sessions.
- **PPO trained in the fitted model** from the same rig data (offline): separates 'having a model' from 'how it is optimised'.
- Report rig minutes, wall-clock and compute time, the learning curves, final quality, smoothness and interventions; say openly that ODIL's recipe was tuned over many versions in simulation.

8. Part 2 in pictures: the real labyrinth

The ODIL path tracker follows the board's printed route at 2.5 cm/s. The route, the holes and a wall map are extracted from a photo of the board; a planner can also find the safest line through the corridors. The rig completed the real maze 3 times (first: 172 s); a typical run reaches about a quarter of the route, and most falls come from arriving at bends 2-3x too fast.

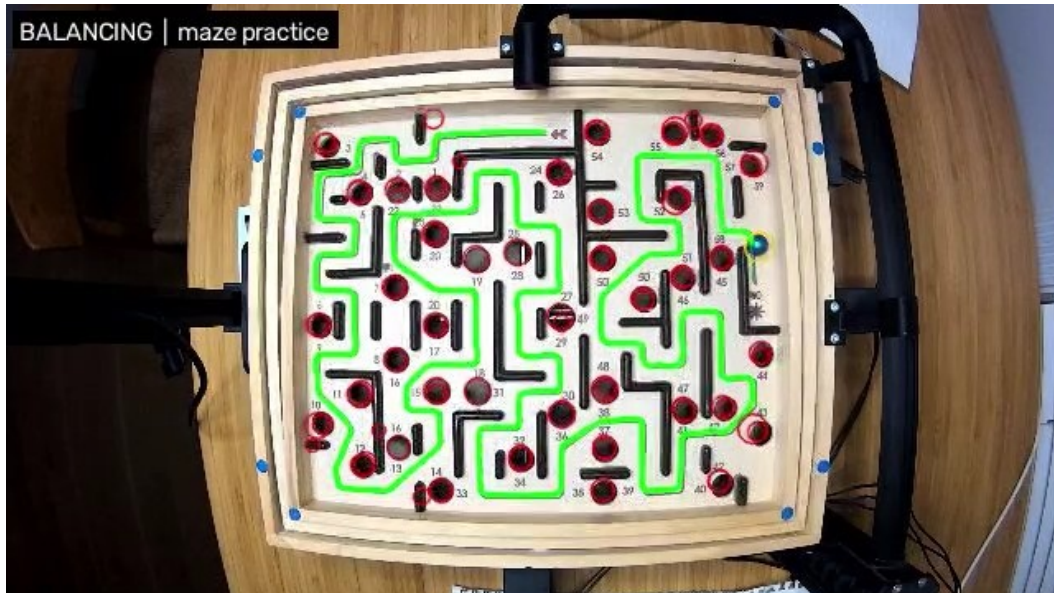


Figure 13. End of the first complete run: the whole route green, the ball at the finish.

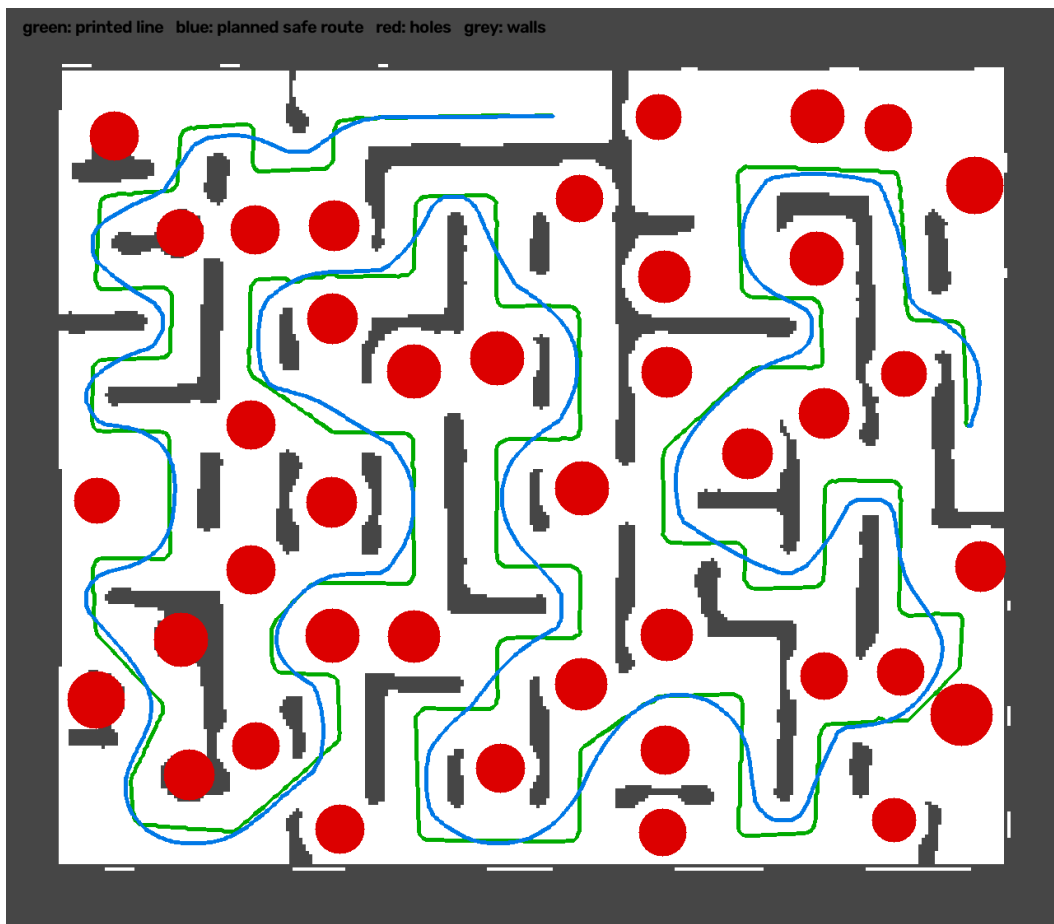


Figure 14. Walls (grey), holes (red), the printed route (green) and the planned safe route (blue).